Python Cheatsheet

Teaching Week 01 to 12 | The R sheet carries the same sections in the same order.

The R cheatsheet has the same sections in the same order, so the other call is found by reading across. The boxes form one script on `heart.csv`, the table Class 01 opens with, run in order from the folder that holds it. A comment that starts with `differs` marks a line whose result is different in the other language, and says why. A script shows only what is passed to `print()`.

1 Loading Packages

Install pandas, numpy, matplotlib, seaborn, scipy and scikit-learn once with `python -m pip install`. Every name the later boxes use is imported here.

```
from importlib.metadata import version
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy.cluster.hierarchy import fcluster, linkage
from scipy.spatial.distance import pdist
from scipy.stats import median_abs_deviation
from sklearn.cluster import DBSCAN, KMeans
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (adjusted_rand_score, classification_report,
    confusion_matrix, mean_absolute_error, r2_score, roc_auc_score, roc_curve,
    root_mean_squared_error, silhouette_score)
from sklearn.model_selection import (GridSearchCV, StratifiedKFold,
    cross_val_score, train_test_split)
from sklearn.neighbors import KNeighborsClassifier, LocalOutlierFactor, NearestNeighbors
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

2 Reading and Inspecting a Table

Check the shape, the names and the type of every column before any statistic. With `include="all"`, `describe()` counts the text columns too.

```
heart = pd.read_csv("heart.csv")
print(heart.shape)                                  # (918, 12)
print(heart.head())                                 # the first five rows
print(heart.dtypes)                                 # storage type per column
print(heart.describe(include="all"))                # all columns; text as counts
print(heart["HeartDisease"].value_counts().sort_index())
```

3 Summary Statistics and Unusual Values

Both languages divide the variance by $n - 1$ and default to type 7 quartiles, linear interpolation in pandas, so these numbers agree. The 1.5 IQR fence catches the 172 impossible cholesterol readings of 0 and 11 high ones.

```
x = heart["Cholesterol"]                                # one column, to keep the calls short
print(x.mean(), x.median())                             # 198.7996 223.0
print(x.var(), x.std())                                 # 11964.89 109.3841, ddof=1
print(pd.Series([2, None, 4]).mean())                   # 3.0; NaN is skipped by default
print(x.quantile([0.25, 0.75], interpolation="linear"))
q1, q3 = x.quantile([0.25, 0.75]); iqr = q3 - q1
print(iqr, x.min(), x.max())                            # 93.75 0 603
out = x[(x < q1 - 1.5 * iqr) | (x > q3 + 1.5 * iqr)]
print(len(out))                                         # 183 = 172 zeros + 11 high values
# no hinge function; matplotlib and seaborn boxplots use type 7 quartiles
print(heart["Age"].corr(heart["MaxHR"]))                # -0.3820447
```

4 Cleaning and Missing Values

Count the missing values before dropping or filling any. The same rows print one lower in Python, because R counts positions from 1 and pandas labels rows from 0.

```
d = heart.copy()                                        # clean a copy; heart stays as read
zero = ["RestingBP", "Cholesterol"]                     # 0 is not a possible reading of either
d[zero] = d[zero].replace(0, np.nan)
print(d.isna().sum())                                   # RestingBP 1, Cholesterol 172, others 0
print(d.index[d["Cholesterol"].isna()][:3])             # 293 294 295, labels from 0
print(len(d.dropna()))                                  # 746 rows with nothing missing
d["was_missing"] = d["Cholesterol"].isna()              # flag before filling
d["Cholesterol"] = d["Cholesterol"].fillna(d["Cholesterol"].median())
print(pd.Series([" North ", "north", "SOUTH"]).str.strip().str.lower().tolist())
print(pd.to_datetime("2026-09-15", format="%Y-%m-%d"))
print(d.duplicated(keep=False).sum())
```

5 Reshaping, Encoding, Scaling, Binning, and Sampling

`scale()` divides by the sample standard deviation and `StandardScaler` by the population one, so the scaled values differ by a factor of $\sqrt{918/917}$. Both binning calls give the same counts.

```
long = heart.melt(id_vars=heart.columns.drop(["RestingBP", "MaxHR"]),
                  var_name="measure", value_name="value")
print(long.shape)                                       # (1836, 12), two rows per patient
print(pd.get_dummies(heart["ChestPainType"], dtype=int).head(3))
sc = StandardScaler().fit(heart[["Age", "MaxHR"]])      # differs, divides by the n sd
print(sc.scale_)                                        # 9.427478 25.446463
# equal width, then equal frequency from the quartile edges
print(pd.cut(heart["Age"], 4).value_counts().sort_index())
q = heart["Age"].quantile([0, 0.25, 0.5, 0.75, 1])
print(pd.cut(heart["Age"], q, include_lowest=True).value_counts().sort_index())
# differs, as the same seed draws other rows in R
s = heart.sample(n=100, random_state=3141)
```

6 Plotting

Give a histogram explicit edges, because the default bins differ and only fixed edges give the same counts in both languages. Each plot opens its own figure with `plt.subplots()`.

```
edges = np.arange(25, 81, 5)                          # the same bins in both languages
fig, ax = plt.subplots(); ax.hist(heart["Age"], bins=edges)
fig, ax = plt.subplots(); sns.boxplot(data=heart, x="Sex", y="Age", whis=1.5, ax=ax)
# differs, as bw_method scales each group's sd while R's bw = 5 is in bpm
fig, ax = plt.subplots(); sns.kdeplot(data=heart, x="MaxHR", hue="Sex", ax=ax)
g = sns.relplot(data=heart, x="Age", y="MaxHR", hue="HeartDisease", col="Sex",
                alpha=0.7, height=4, aspect=0.875)     # a 0/1 hue gets two colours
g.set_axis_labels("Age (years)", "Maximum heart rate (bpm)")
Path("results").mkdir(exist_ok=True)                  # savefig() needs the folder
g.savefig("results/age-maxhr.pdf")
print(np.histogram(heart["Age"], bins=edges)[0])      # 4 17 59 ... 24 7
fig, ax = plt.subplots(); ax.scatter(heart["Age"], heart["MaxHR"])
```

7 Transforming and Joining Tables

Check the row count after every join. A key repeated in the lookup table would multiply the rows without an error, and the many-to-one check stops it there.

```
summary_table = (heart
    .query("Age >= 40")                               # keep the rows that qualify
    [["Age", "Sex", "MaxHR", "HeartDisease"]]         # keep the columns you use
    .assign(band=lambda t: np.where(t["Age"] >= 60, "60+", "40-59"))
    .groupby(["Sex", "band"], as_index=False)         # one group per Sex and band
    .agg(n=("Age", "size"), mean_maxhr=("MaxHR", "mean"))
    .sort_values("mean_maxhr", ascending=False))      # sort the summary, largest first
print(summary_table)                                  # 4 rows, F 40-59 first
lookup = pd.DataFrame({"ChestPainType": ["ASY", "ATA", "NAP", "TA"],
                       "pain": ["none", "atypical", "non-anginal", "typical"]})
joined = heart.merge(lookup, on="ChestPainType", how="left",
                     validate="many_to_one")
assert len(joined) == len(heart)                      # a left join keeps every row
summary_table.to_csv("results/summary.csv", index=False)
```

8 Reproducible Reports

Render from a fresh session with the command shown, so the report holds what the source computes rather than what an interactive session remembers.

```
# in report.ipynb, display(Markdown(f"The mean age is {heart['Age'].mean():.1f} years."))
print(f"{heart['Age'].mean():.1f}")                   # 53.5, what the sentence shows
print(summary_table.round(3).to_string(index=False))
# jupyter nbconvert --execute --to pdf --no-input report.ipynb
print(version("pandas"))                              # versions, for the reproduction record
```

9 Splitting and Scoring Predictions

The two languages put different rows in the test set, so every score from here on differs between the sheets. Two rules stand in for models, MaxHR below 140 for disease and 220 minus age for MaxHR.

```
num = ["Age", "MaxHR", "Oldpeak"]                       # the predictors from here on
X, y = heart[num], heart["HeartDisease"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.30,
    random_state=3141, stratify=y)                      # differs, other rows than R
pred = (X_te["MaxHR"] < 140).astype(int)
print(confusion_matrix(y_te, pred))                     # truth down the rows
# precision, recall and F1 for each class
print(classification_report(y_te, pred, digits=4))
fpr, tpr, thr = roc_curve(y_te, -X_te["MaxHR"])
print(roc_auc_score(y_te, -X_te["MaxHR"]))              # 0.7383
hr, p = X_te["MaxHR"], 220 - X_te["Age"]                # a rule for maximum heart rate
print(mean_absolute_error(hr, p))                       # MAE 31.57
print(root_mean_squared_error(hr, p))                   # RMSE 39.11
print(np.corrcoef(hr, p)[0, 1] ** 2)                    # 0.136, caret's Rsquared
print(r2_score(hr, p))                                  # -1.148, 1 - SSres/SStot
```

10 Four Models

Each model learns from the training rows only, scaling included, and is scored on the test rows.

```
knn = make_pipeline(StandardScaler(),
                    KNeighborsClassifier(n_neighbors=5))
# KNeighborsClassifier breaks ties toward the lowest class
knn.fit(X_tr, y_tr)                                     # the scaler learns X_tr only
print(knn.score(X_te, y_te))                            # test accuracy 0.728
lm = LinearRegression().fit(X_tr[["Age"]], X_tr["MaxHR"])
print(lm.intercept_, lm.coef_)                          # intercept 191.65, slope [-1.02]
print(mean_absolute_error(X_te["MaxHR"], lm.predict(X_te[["Age"]])))
lda = LinearDiscriminantAnalysis().fit(X_tr, y_tr)
print(lda.score(X_te, y_te))                            # 0.764
svm = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0, gamma="scale"))
print(svm.fit(X_tr, y_tr).score(X_te, y_te))            # 0.75; SVC does not scale
fig, ax = plt.subplots(); ax.scatter(X_te["MaxHR"], lm.predict(X_te[["Age"]]))
ax.axline((0, 0), slope=1)                              # predicted against observed
```

11 Decision Trees and Pruning

`cp` is α over the root error and `ccp_alpha` is α over the training rows on Gini impurity, so neither carries over. By default `rpart` stops at `cp = 0.01` and scikit-learn grows to purity.

```
cv10 = StratifiedKFold(n_splits=10, shuffle=True, random_state=3141)
kw = dict(min_samples_split=20, min_samples_leaf=7, random_state=3141)
path = DecisionTreeClassifier(**kw).cost_complexity_pruning_path(X_tr, y_tr)
print(len(path.ccp_alphas))                             # 36 subtrees, no CV error yet
gs = GridSearchCV(DecisionTreeClassifier(**kw), {"ccp_alpha": path.ccp_alphas}, cv=cv10)
pruned = gs.fit(X_tr, y_tr).best_estimator_             # best mean CV accuracy
plot_tree(pruned, feature_names=num)
print(pruned.feature_importances_)                      # Age 0.075, MaxHR 0.289, Oldpeak 0.636
print(pruned.score(X_te, y_te))
```

12 Resampling and Tuning

The best cross-validated score is a maximum over the grid, so it is optimistic. Report the test score with a bootstrap interval, or nested cross-validation.

```
cv5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=3141)
grid = {"kneighborsclassifier__n_neighbors": [1, 5, 9, 15]}
rng = np.random.default_rng(3141)
gs = GridSearchCV(knn, grid, cv=cv5)                    # knn scales inside each fold
gs.fit(X_tr, y_tr)
print(gs.best_params_)                                  # n_neighbors = 9
print(gs.best_score_)                                   # 0.746, selected, so not an estimate
hit = (gs.predict(X_te) == y_te).to_numpy()
print(hit.mean())                                       # 0.728, the estimate (X_te used once)
acc = [rng.choice(hit, len(hit)).mean() for _ in range(2000)]
print(np.percentile(acc, [2.5, 97.5]))                  # 0.674 0.779, a 95% interval
print(cross_val_score(gs, X, y, cv=cv5).mean())         # 0.732, nested CV
```

13 Distances and Clustering

Scale first, because a distance is in the units of the columns. With the sample standard deviation in both languages, k-means reaches the same three clusters, numbered differently.

```
Z = ((heart[num] - heart[num].mean()) / heart[num].std(ddof=1)).to_numpy()
D = pdist(Z, "euclidean")
# Gower distance has no scipy or scikit-learn call; the notes write it out
# differs for some k, as scikit-learn runs Lloyd and R runs Hartigan-Wong
print([KMeans(k, n_init=50, max_iter=100, random_state=3141).fit(Z).inertia_
       for k in range(1, 9)])
km = KMeans(n_clusters=3, n_init=50, max_iter=100, random_state=3141).fit(Z)
print(np.bincount(km.labels_), km.inertia_)             # [238 317 363] 1386.006
print(silhouette_score(Z, km.labels_))                  # 0.2946317
hc = fcluster(linkage(Z, method="complete"), 3, criterion="maxclust")
print(np.bincount(hc)[1:])                              # differs, 76 624 218 (tied distances)
nn = NearestNeighbors(n_neighbors=5).fit(Z)             # column 0 is the point itself
fig, ax = plt.subplots(); ax.plot(np.sort(nn.kneighbors(Z)[0][:, -1]))
db = DBSCAN(eps=0.5, min_samples=5).fit(Z)
print(np.bincount(db.labels_ + 1))                      # 78 noise (-1), then 824 4 7 5
print(adjusted_rand_score(km.labels_, hc))              # differs, 0.190
```

14 Outliers

The 1.5 IQR fence is in Section 3. The median and the MAD resist the masking that hides a group of large values from it, and LOF ranks rows by local density, with no universal cut-off.

```
# n_neighbors=4 leaves the point out, as minPts = 5 does not; sklearn stores -LOF
lof_score = -LocalOutlierFactor(n_neighbors=4).fit(Z).negative_outlier_factor_
print(np.argsort(-lof_score)[:3])                       # 325 332 352, the same rows from 0
x = heart["Cholesterol"].to_numpy(float)                # MADs from the median
r = np.abs(x - np.median(x)) / median_abs_deviation(x, scale="normal")
print((r > 3).sum(), np.argsort(-r)[:3])                # 180, 172 of them zeros; 149 616 76
```